tuning pipeline in the eight steps shown in Figure 1.

Step 1: Environment setup

Install and configure all the dependency stacks of Hugging Face and TRLs (technology readiness levels) required for PEFT (Parameter-Efficient Fine-Tuning) using LoRA and QLoRA:

```
# Keep it simple: use current Colab PyTorch + CUDA + NumPy
!pip install -q --upgrade pip

# Install latest Hugging Face + TRL stack that supports NumPy
2.x
!pip install -q \
  transformers \
  accelerate \
  datasets \
  trl \
  peft \
  bitsandbytes \
  einops \
  wandb
!pip install -U bitsandbytes
```

Step 2: Initialisation and essential configuration

This configuration block initialises all essential hyperparameters and environment settings required for QLoRA-based fine-tuning of the Mistral-7B-Instruct model. It establishes the base model path, output directory for checkpoints, and key training parameters.

```
import trl, transformers
import importlib, pkgutil
print("trl:", trl.__version__)
BASE_MODEL = 'mistralai/Mistral-7B-Instruct-v0.2'
OUTPUT_DIR = '/content/qlora-mistral-sysprog'
USE_4BIT = True  # QLoRA
SEQ_LEN = 1024   # keep <= 2048 for T4 comfort
EPOCHS = 10       # increase for real training
LR = 2e-4
BATCH_SIZE = 1
GRAD_ACCUM = 8
WARMUP_RATIO = 0.03
SAVE_STEPS = 50
LOG_STEPS = 1
VAL_SET_SIZE = 0  # set >0 if you add a validation split
```

We have described the key training parameters, which can be initialised and updated as per your use case and fine-tuning requirements.

USE_4BIT=True(QLoRAenablement): Flag to enable quantisation; this will be used to activate 4-bit quantization and QLoRA enablement.

SEQ_LEN=1024: SEQ_LEN is the maximum number of tokens the training pipeline feeds to the model in a single sample (per forward/backward pass). Choose SEQ_LEN as per your data set size and VRAM limit. For fine-tuning of this particular use case on T4, SEQ_LEN size of 1024 is chosen.

EPOCHS=8: An epoch is one complete cycle through the entire training dataset. Since adapters have relatively few trainable parameters, they converge quickly; 3-5 epochs are often sufficient for narrow domains. Use 8-10 only if the dataset is small and regularisation is in place (dropout, early stopping). Choose the epoch count as per the distribution of your training dataset samples; too many epochs can lead to model overfitting.

LR = 2e-4: Learning rate (LR) is a fundamental hyperparameter in gradient-descent-based optimisation. It controls how large a step the model takes when updating its trainable parameters (weights, biases, or LoRA adapter matrices) based on the computed gradients. With LoRA/QLoRA, slightly higher LR than full-fine-tuning is typical; we have taken LR 2e-4 Safe grid: {1e-4, 2e-4} for general SFT.

BATCH_SIZE = 1 (per device): Batch size refers to the number of training samples the model processes in a single forward + backward pass before updating the parameters (in this case weight and bias of LoRA adapters). Here we have taken a tiny batch size of 1 per device and used gradient accumulation. GRAD_ACCUM =8 accumulates gradients over 8 micro-batches to emulate a larger effective batch while staying within VRAM. Effective batch ≈ BATCH_SIZE × GRAD_ACCUM. Typical sweet spot on T4 is 8-16 when SEQ_LEN ≤ 1024

WARMUP_RATIO = 0.03: Warmup ratio is the fraction of total training steps during which the learning rate (LR) is gradually increased from 0 up to its target value. We have taken a 3% warmup of total steps to help stabilise early updates with low-precision weights. Use an appropriate warmup ratio as per your dataset and use case.

SAVE_STEPS=50,LOG_STEPS=1: Frequent logging is handy for debugging early instability. On Colab, saving too often increases I/O overhead; if training is slow, consider SAVE_STEPS=200-500 once stable.

Step 3: Data preparation and transformation for instruction tuning

For this blueprint, synthetic training samples were generated in JSONL format; however, for real-world fine-tuning workflows, you should curate or collect domain-specific datasets representative of your target use cases.

```
{"instruction": "Provide a minimal C snippet that demonstrates
the POSIX mmap() usage with proper error handling.", "input":
```